Projektuppgift

DT211G – Frontend-baserad utveckling

Projektuppgift
Dynamisk webbplats med mashup

Isak Söderkvist

Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Isak Söderkvist, isso2500@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

Målet med detta projekt är att skapa en dynamisk webbplats med mashup. Webbplatsen ska bestå av HTML, SCSS och Javascript, samt kallande till olika APIer som ska kommunicera med varandra och skapa ett sammansatt innehåll på webbplatsen.
Webbplatsen, Apöllo 11, använder sig av APIer som tillhandahåller information om den Internationella Rymdstationens nuvarande position i latitud och longitud och platsinformation om bryggerier i världen.
Informationen från APIerna behandlas och omvandlas till läsbar text som visas på webbplatsen samt visuellt på en karta, med markörer för dem olika platserna.
Kartan tillhandahålls av Open-Source tjänster, som står för själva kartfunktionen och en som står för grafiken.

Nyckelord: API, Mashup, SCSS, JavaScript

Innehållsförteckning

Sammanfattning **iii**

Förord **iv**

Terminologi **vi**

1 Introduktion / Inledning **1**
1.1 Bakgrund och problemmotivering 1
1.2 Övergripande syfte / Högnivåproblemformulering 2
1.3 Avgränsningar 2
1.4 Konkreta och verifierbara mål / Detaljerad problemformulering 2
1.5 Översikt 3
1.6 Författarens bidrag 3

2 Teori / Bakgrundsmaterial **4**
2.1 Definition av termer och förkortningar 4
2.1.1 Exempel på rubriknivå 3 5
2.2 Att referera eller citera 5
2.3 Källförteckning och källhänvisningar 5
2.4 Automatiskt numrerade källhänvisningar 6
2.5 Illustrationer 7
2.6 Matematiska formler 8

3 Metod **9**

4 Konstruktion / Lösningsalternativ **10**

5 Resultat **11**

6 Slutsatser / Analys / Diskussion **12**

Källförteckning **13**

Bilaga A: Dokumentation av egenutvecklad programkod **14**
Exempel på underrubrik 14

Terminologi

Akronymer/Förkortningar

SCSS	Sassy CSS, en pre-processor för CSS som kompilerar och effektiviserar vanlig CSS.
API	Application Data Interchange, en anslutning mellan program/webbtjänster som erbjuder olika tjänster.

1 Introduktion

Rymden och stjärnorna har under civilisationens historia spelat en otroligt stor roll i samhället. Inte bara något fint och lugnande att titta på när natten faller utan något som ger mening för dem som tror på något större, som tror på horoskop och jordbrukare innan den stora, etablerade kalendern som vi har idag. Efter teknologisk utveckling har människan äntligen möjlighet att befinna sig uppe bland stjärnorna och ett nytt problem uppstår då: Tillgången till fatöl.

Alkohol, i detta fall öl, har också spelat en otroligt stor roll inom mänskligheten historia, kanske inte exakt samma roll idag som den hade för egyptierna 5000 år sedan. Jag ville skapa ett sätt för våra stjärnfarare att hitta till närmaste fatöl ifall suget någonsin uppstår. En lättillgänglig webbplats som med ett knapptryck kan visa den närmsta vägen till bryggeriet.

Webbplatsen ska hämta information från separata APIer, slå ihop denna information och visa den på webbplatsen i läsbart format. Webbplatsen ska kunna hämta in ny information utan att laddas om från samma APIer vid knapptryck.
Ett problem med användarinteraktion och API-hämtningar är att den information som man vill visa kanske inte finns där när man trycker på en knapp eller andra menyer. Då måste man implementera kod som väntar på att informationen faktiskt finns där.

Det ska inte heller gå att få fram mer resultat än ett per hämtning. Varje knapptryck ska starta funktionen och avvakta tills resultatet finns, och endast visa ett resultat.

APIer som tillhandahåller information via arrayer och objekt innehåller ofta väldigt mycket information, både relevant och irrelevant för uppgiftens syfte. Webbplatsens kod ska effektivt kunna filtrera ut den onödiga informationen, eller effektivt endast plocka ut den värdefulla informationen. För rymdstationens API finns det flera attribut som tidszon och hastighet vilket jag inte är intresserad av. För bryggerierna kan det finnas information så som id (för katalogisering i APIet), flera olika adresser, webbplats URL mm. Beroende på vad man vill uppnå kan dessa värderas som onödiga.

Informationen kan också komma i fel ”format” för att kunna bearbetas av kodhanterarna. Värden så som siffror kan lagras som *floats* eller *integers*, alltså siffror, eller som strängar, som består av bokstäver. För oss människor finns inte samma sorts krav. Läser vi ett tal vet vi att det är ett tal och hur vi ska behandla det. Men en kodredigerare som förväntar sig ett tal men får siffran 15 som en sträng kommer få error.

2 Teori

Vite har använts under hela arbetsprocessen och fungerar som både en simulator och automatiserar kompileringen och skapandet av ett projekt. När man arbetar med projekt har man s.k källkodsfiler och publiceringsfiler. Källkodsfilerna i kodhanteraren är mänskligt strukturerade, läsbart för den som arbetar med det och kanske, för tydlighetens skull, lite för många kataloger. Men det fungerar, eftersom allting sker lokalt hittills och man som utvecklare inte har kravet på sig själv att allt ska fungera otroligt sömlöst.
Vite optimerar filerna, genom att slå ihop dem, ta bort rader och mellanrum och annat onödigt som bara är till för läsbarheten, och paketerar dem. Detta går också att göra manuellt, men vore otroligt tidskrävande. Använder man plugins så som optimisering för bilder så sköter Vite även den optimeringen och paketerar allt så litet som möjligt.

Den teknik av API användning som förekommer i detta projekt är något som kallas Mashup [1]. APIer ger mycket information som går att använda, men det är oftast endast för en viss specifik sak som man får informationen om. Man kan skapa en webbplats som kallar på olika kattraser som man vill adoptera från ett katthem, men vill man få fram vart närmsta katthem finns så måste man, förutom att surfa till Google Maps, implementera ett API som har den informationen. Att hämta information från olika APIer och slå ihop olika delar från dem för att presentera ny information kallas Mashup.

Karttjänsterna för webbplatsen har också tagits fram av olika tjänster, som samarbetar för att få fram ny sammansatt information. I det här fallet finns det en tjänst som ger själva funktionerna för en karta, så som markörer, förflytta bilden, klicka, zooma osv. Men den ger inte den funktionalitet som kallas ”tiles”, det som faktiskt visar ritningarna och topologin av området man vill se. Tänk som att en tjänst ger dig ramen kring en sandlåda med grävskopa, spadar och hinkar och en annan tjänst ger dig sanden inuti lådan. Dem fungerar inte utan varandra.

3 Metod

För att slå ihop mina APIer och samtidigt leverera dem i funktioner som väntar på att informationen finns där används funktioner med *async/await*. [2] När man definierar en funktion kan man ange den som *async*, vilket möjliggör användningen av *await* inuti funktionen. Med dessa två kan man skapa en funktion som börjar köras, men väntar på att något annat uppfylls innan den fortsätter med sin kod.

För att faktiskt visa informationen på webbplatsen, så som bryggeriets plats och rymdstationens latitud och longitud används DOM (Document Object Model) [3]. Med DOM kan man med JavaScript manipulera HTML-element i webbplatsen genom att skapa, radera, ändra och ändra plats.

Genom att stänga av knappen som kallar på informationen vid knapptryck stoppas risken av flera hämtningar. Webbplatsen skickar inte onödiga förfrågningar som kan överväldiga en server eller överväldiga själva webbplatsen när alla svar kommer tillbaka. Knappen slås sedan på igen när funktionen körts igenom och man kan klicka igen för att hämta in nytt resultat.

Onödig information som jag inte behöver för webbplatsen filtreras lätt ut med variabler. Dessa går att kalla på direkt i en funktion utan att skapa en variabel av det, men det blir svårare att hålla reda på och mer text i slutändan, vilket påverkar prestandan.

Formatet av datan är inte heller ett större problem, speciellt inte när jag endast behöver informationen för ett syfte i slutändan. Vore det så att det finns situationer där data måste vara en sträng eller integer blir det lite mer arbete, men i mitt fall kommer webbplatsen endast använda sig av floating integers, dvs siffror med decimaler.

4 Konstruktion

Det första som sker på webbplatsen är att kartan skapas. I funktionen som skapar kartan har jag lagt till statiska koordinater för Houstons rymdcentral som startpunkt, eftersom det passar tematiskt till syftet med webbplatsen. Jag övervägde först att be om användarens plats och starta därifrån, men det var inte nödvändigt att be om platsen eftersom den ändå skulle ändras. Och tanken är att besökaren i praktiken ska vara en astronaut ombord på rymdstationen. Med laddningen av webbplatsen skapas även en s.k ”event-listener” på en knapp i startsidan som i sin tur, vid klickning, hämtar in datan från APIerna.

I stället för att datan hämtas in på direkten valde jag att göra fetch beroende av knapptryck. Jag vill inte att webbplatsen ska påverkas negativt prestandamässigt utan att besökaren själv ”går med” på det. Webbplatsen hade varit långsammare att besöka, speciellt med långsammare nätverksuppkoppling. En webbplats som är otydligt och tar tid att ladda in riskerar att bli ignorerad av besökare [4]. Detta uppnås med nedan, i figur 1, där en eventlistener väntar på att allt är inladdat och lägger sedan till en eventlistener på knappen, som i sin tur hämtar datan:

```
3   /**
4    * Eventlyssnare för DOMContent, som kallar på funktionen fetchData
5    */
6   document.addEventListener("DOMContentLoaded", () => {
7      var map = createMap(L.map("map"))
8
9
10     document.getElementById("beerBtn").addEventListener("click", () =>
11        fetchData(map))
12
13  })
14
15  /**
16   * Funktionen väntar på att kartan skapas i DOMContentLoaded och kör sedan async/await från APIerna.
17   * @param {var} map
18   * @returns spaceCoords (ISS position), brewCoords (Närmsta bryggeri), map(kartan), brewName (Brygge
19   */
20 > async function fetchData(map) {···
58  }
```

figur 1

Detta innebär också att felaktig inhämtning av data undviks. Knappen som hämtar in datan får också sin egenskap ändrad till *disabled* under den tid som funktionen körs. Se figur 2 och 3. Innan knappen är tryckt har den sin animation spelande som vanligt och är lite ljusare. Skillnaden när funktionen körs är att knappen inte längre fungerar, och blir mörkare för att markera att den inte är aktiv just nu.

figur 2

figur 3

I samband med att informationen endast laddas in när en knapp blir tryckt så påverkar det också vad som är möjligt på webbplatsen. Med async/await funktioner kan jag se till att informationen endast visas när den är hämtad och tillgänglig. Jag implementerade detta genom att skapa en knapp med en eventlyssnare som kör en funktion. Funktionen kallar på APIerna, skapar variabler och filtrerar vid behov och i sin tur kallar sedan på funktioner som tar värden och ändrar kartan, skapar markörer och fyller på texten jag vill visa. Dessa funktioner kan granskas i bilaga A, längst ned.

5 Resultat

För denna webbplats finns det fem mål som ska uppfyllas gällande funktionaliteten för webbplatsen:

- Webbplatsen ska hämta information från separata APIer, slå ihop denna information och visa den på webbplatsen i läsbart format.
- Eftersom datan måste hämtas externt dröjer det innan informationen finns på plats. Webbplatsen ska kalla på data och vänta på att datan har tagits emot.
- Varje hämtning ska endast visa ett resultat. Flera resultat per hämtning eller flera klick ska inte resultera i mer än ett resultat.
- Endast relevant information ska filtreras ut från APIerna och dess innehåll. Hastighet, tidszon och lokal tid till exempel är inte av intresse.
- Text och siffror ska omvandlas till rätt format för att kodhanteraren ska kunna bearbeta saker som latitud och longitud ordentligt.

Webbplatsen kallar på APIer som tillhandahåller information om den Internationella rymdstationen och bryggerier runtom i världen. Dessa får variabler tilldelade och skrivs sedan ut på webbplatsen med hjälp av DOM, som manipulerar HTML element med ID och klasser. Webbplatsen skapar <li> element i en onumrerad lista som ger information om rymdstationens nuvarande latitud och longitud, samt närmsta bryggeri med namn och stad. I samband med texten ändras även kartans egenskaper vid klick. Kartan ändrar position som inkluderar rymdstationens och bryggeriets platser, och placerar ut markörer för att visa dem.

APIerna tillkallas inte automatiskt utan endast när besökaren klickar på *beerBtn*, eller ”Klicka här för öl!” knappen. Till knappen är en async funktion kopplad som endast skapar variabler och behandlar datan när den fått svar av APIerna. Detta innebär att ogiltiga data inte kan tas emot, är det så att APIerna inte svarar får man annat fel.

Varje knapptryck hämtar ett bryggeri, och den nuvarande positionen av rymdstationen. Önskar man ha ett nytt resultat behöver man klicka på knappen igen, och då raderas tidigare resultat och byts ut med ett nytt. Flera klick på knappen under laddningstid går inte då den får klassen *disabled* och vägrar fler klick.

Ur APIerna sorteras information ut. Som tidigare nämnt finns det information om rymdstationen så som tidszon och hastighet, men det tas inte med eftersom målet är att endast visa platsen. Med bryggerierna tas endast namn och stad med, och kartan byter vy så bägge platser syns.

Datans format omvandlas till *floats* (floating integer, siffror med decimaler) för att säkerställa att funktioner som behandlar arrayer fungerar.

6 Slutsatser

Dem ursprungliga kraven för denna webbplats uppfylls, men kanske inte perfekt i alla aspekter som jag önskat. Problemen uppstår bland bryggeriets API. Under utvecklingen upptäckte jag att det finns vissa fel i APIn så som koordinater och platser där det logiskt sett borde finnas bryggerier. Jag har förståelse för att det troligen är näst intill omöjligt att dokumentera alla bryggerier i världen och att den katalogen skulle bli otroligt stor. Och det är ingen kritik mot dem som tagit sin egen fritid att utveckla något sådant här, men det ledde till att sökfunktionen inte alltid var helt korrekt, både att den ibland markerade ut fel positioner och att den inte markerade platser alls. Jag tror att det är en nackdel man får förvänta sig när det kommer till APIer utvecklade av privatpersoner som hobby, speciellt när det handlar om en sådant omfattande tema som världens bryggerier.

Oförväntat nog så upplevde jag också problem med APIn för rymdstationen. Den ursprungliga jag använde mig av slutade fungera i mitten av utvecklingen, vilket ledde till att jag var tvungen att byta API. Det var relativt snabbt fixat då formatet var likt och parametrarna var lika.

Utöver dessa fel är jag nöjd med webbplatsen. Det är ett väldigt nischat tema, som man i praktiken egentligen aldrig kommer kunna använda sig av, om man inte har en astronaut vän som kan hoppa fallskärm till närmsta bryggeri.
Alla mål som jag hade satt upp i början har uppnåtts, på en tillfredsställande nivå. Att hämta in positioner och jämföra deras avstånd, både med kod och kunna visa upp det på en karta för att verkligen visuellt visa det är spännande.

Jag kom även i kontakt med utvecklaren för APIet för bryggerierna och man kan själv lätt uppdatera APIn med hjälp av git och pull-requests, så det finns möjlighet att själv kunna bidra till projektet och fixa dem problem jag nämnde i början av detta kapitel. Det är något jag absolut kan tänka mig som ett hobbyprojekt, och bygga på webbplatsen med flera APIer eller kanske mer interaktion som att visa bryggeriets husöl eller liknande.

Källförteckning

Här följer exempel på hur en källförteckning kan utformas enligt Vancouver-systemet. Den är automatiserad enligt metoden numrerad lista och korsreferenser, som beskrivs i kapitel **Fel! Bokmärket är inte definierat.**.

[1] IBM, *"API Mashups"*,
https://www.ibm.com/docs/en/wm-apigateway-ipaas?topic=implementation-api-mashups
Reviderad 2026-01-20. Hämtad 2026-03-08

[2] Mozilla Developer Network, "*async function*"
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
Reviderad 2025-07-08, Hämtad 2026-03-09

[3] Wikipedia, "*Document Object Model*"
https://en.wikipedia.org/wiki/Document_Object_Model
Reviderad 2026-03-05. Hämtad 2026-03-09

[4] Pneumamedia, *"The 5 second rule of web design"*
https://www.pneumallc.com/blog/the-5-second-rule-of-web-design
Reviderad 2022-16-03. Hämtad 2026-03-09

Bilaga A: Dokumentation av egenutvecklad programkod

```
 20 > async function fetchData(map) {···
 58   }
 59
 60
 61   /**
 62    * Placerar ut markörer utefter koordinater från fetch av iss-position och OpenBreweryDB
 63    * @param {int} spaceCoords
 64    * @param {int} brewCoords
 65    * @param {var} map
 66    */
 67
 68   var brewMarker;
 69   var spaceMarker;
 70   var path;
 71 > function placeMarkers(spaceCoords, brewCoords, map) {···
108   }
109
110   /**
111    * Flyttar kartan till markörernas plats.
112    * @param {float} spaceCoords
113    * @param {float} brewCoords
114    * @param {var} map
115    */
116 > function showMarkers(spaceCoords, brewCoords, map) {···
119   }
120   /**
121    * Skapar karta med Lyndon B Johnson Space Center centrerat.
122    */
123 > function createMap(map) {···
147   }
148
149
150
151   /**
152    * Lägger till koordinaterna i en lista, ovanför kartan.
153    * @param {string} spaceCoords
154    * @param {string} brewName
155    * @param {string} brewCity
156    */
157 > function addToList(locationInfoSpace, spaceCoords, locationInfoBrew, brewName, brewCity, locationInfoEl) {···
163   }
```